

Optimization of SLMs for Efficient Code Generation

HPML Final Project — Columbia University, Spring 2026

Jasmine Truong
Columbia University
New York, NY, USA
jt3595@columbia.edu

Yingxin Zhang
Columbia University
New York, NY, USA
yz3202@columbia.edu

Jianyi Gao
Columbia University
New York, NY, USA
jg4553@columbia.edu

Arnav Mahajan
Columbia University
New York, NY, USA
am6442@columbia.edu

Abstract—As LLMs are increasingly utilized for code generation, most training and finetuning done focuses on correctness often overlooking runtime efficiency. In this paper, we leverage existing work done in other research papers in order to optimize our LLMs against runtime efficiency alongside the standard correctness metric. We build upon existing work by doing this with SLMs under 3B to maximize speed from code generation to code execution.

I. INTRODUCTION

In the last few years, LLMs have become adopted industry-wide for code-generation tasks. Tools like Cursor, Claude Code, Windsurf, etc. have become critical to developer workflows. Likewise, AI Agents leverage code generation and code executions during their runtimes as a tool, being leveraged to access and operate on API data or manage file systems.

For latency critical codebases, the speed of LLM-generated code begins to matter as much as correctness. For latency-critical applications that leverage AI code generation and execution in their pipelines (such as the AI agents discussed above), the speed of *generation* also becomes a factor alongside execution speed, motivating exploration of efficient inference serving, either through smaller LLMs [1] or inference engine optimization [2].

However, most LLMs’ code generation capabilities are benchmarked for functional correctness (such as passing test suites [3]) rather than runtime efficiency. As LLM Code Generation becomes even more widely used, both in user-facing workflows and latency critical applications, we are motivated to explore runtime efficiency as an objective as important as correctness in both the fine-tuning process and serving layer. In this paper, we build off existing work on runtime-efficient code generation [4] by leveraging a very SLM of Qwen2.5-Coder-1.5B-Instruct [5] in order to **generate efficient code quickly**.

We start off by taking a filtered version of the EffiCoder dataset’s task descriptions to generate multiple sample candidates. Using these, we create a finetuning dataset with a *runtime-aware* objective (of the fastest candidate solution) and a control dataset. The resulting datasets are used to fine-tune Qwen2.5-Coder-1.5B-Instruct for the task of efficient code

generation. QLoRa [6] is leveraged for us to efficiently train our models in a resource efficient manner. We find that the fine-tuning process increases both the Pass @1 by nearly 30% and reduces our avg. execution latency by over 100 ms.

Lastly, we run comprehensive benchmarking to test our solution against various inference engines / serving techniques such as TGI and vLLM [2] and find up to 8.3x improvements in serving throughput through vLLM. Below, we discuss these findings and our methodology in more detail.

II. RELATED WORK

A. Code Generation with LLMs

Recent large language models have been able to achieve substantial success on many code generation benchmark tasks, including HumanEval [7] and MBPP [8]. Many have significantly improved functional correctness in code generation, including CodeBERT [9], AlphaCode [10], and Code Llama [11]. While many of these benchmarks focus only on verifying the correctness of generated programs through correct outputs, they do not account for the execution efficiency of the generated programs. Our work aims to address this problem by optimizing for correct runtime performance in addition to ensuring correctness.

B. Runtime-Efficient Code Generation

EffiCoder [4] is the research project that closely relates to this work. This project provides a collection of coding challenge datasets, reference solutions with associated measured execution runtimes, and uses runtime-aware candidate selection to prioritize faster solutions when selecting candidates during training. As part of our work, we use the EffiCoder datasets and evaluate the impact of both training time and serving time optimizations on throughput with 1.5B parameter models. The need to optimize the entire end-to-end code generation and execution pipeline of latency-constrained AI agents [1] has further motivated this area of research.

C. Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning techniques allow for the adaptation of large models to different tasks with significantly

less computational effort. Techniques such as LoRA [12] provide a low-rank decomposition of existing model weights to allow reductions in the number of parameters that require fine-tuning. QLoRA [6] extends this concept by using 4-bit NF4-based quantization of the frozen base model to enable low cost fine-tuning of 4-bit Adapters. In our work, we use QLoRA with 16 rank Adapters and 4-bit NF4 quantization to fine-tune Qwen2.5-Coder-1.5B-Instruct [5] in a resource efficient manner.

D. Efficient LLM Serving

Autoregressive decoding is a memory bandwidth-bound operation when processing small batch sizes, and the significant impact of this constraint can be viewed through roofline performance modeling techniques [13]. vLLM [2] improves the current state of autoregressive decoding efficiency by using Continuous Batching and Paged Attention techniques to reduce KV-cache memory fragmentation, thus enabling more concurrent sequence uses of memory and improving the overall utilization of the GPU. FlashAttention [14] provides similar improvements to autoregressive memory bandwidth usage by combining the operations in the attention kernel and retaining the intermediate activations in SRAM versus HBM. Our serving benchmarks show the benefits in throughput performance of these techniques.

III. MODELS AND DATA DESCRIPTION

A. Model Architecture

We use Qwen2.5-Coder-1.5B-Instruct [5], a 1.5 billion decoder-only transformer pretrained on 5.5 trillion tokens of code and natural language. The model has 28 transformer layers, a hidden dimension of 1536, 12 query attention heads, and 2 key-value heads via grouped query attention, which reduces KV-cache memory and enables more concurrent sequences under vLLM. It also supports a 128K-token context window.

At BF16 precision, the 1.54B parameters occupy approximately ~ 3 GB of device memory. This means making small-batch inferences is memory-bandwidth-bound. This property motivates the roofline and serving analysis in Section VI.

B. Dataset

We use EffiCoder [4], a data set created to evaluate the efficiency of algorithms and create code for completing programming tasks. The base data set contains ~ 9400 different programming problems. The first step we take is to filter out all problems pertaining to algorithms (i.e. sorting, graph algorithms, dynamic programming, etc), as well as removing noisy data sets (problems that use randomness, threads, API calls or file I/O methods), resulting in a total of ~ 6900 accepted programming problems we are able to use for benchmarking. We then benchmark the reference solution for each problem by executing it a total of 7 times in an isolated subprocess and recording the median execution time (running time). Next, we only keep those problems whose reference solution passes all provided test cases ($\sim 96\%$ pass rate), resulting in the final number of candidate programming problems available for candidate generation and training being 6646.

IV. TRAINING AND PROFILING METHODOLOGY

A. Experimental Setup

Hardware

- 1x NVIDIA A100-SXM4-40GB (HBM2e, 40 GB, 1555 GB/s, 312 TFLOPS BF16)
- CUDA 12.x, Ubuntu 22.04

Software Environment

- Python 3.12, PyTorch 2.10-2.11 (cu128)
- HuggingFace Transformers, PEFT, TRL (SFTTrainer) vLLM (PagedAttention + continuous batching) for serving benchmarks
- Weights & Biases for experiment tracking

B. Training Data Construction

We construct supervised fine-tuning targets from the benchmark candidate pool instead of directly using the raw EffiCoder reference solutions. We first create deterministic training, validation and test splits from the clean 6,646 problem subset using an 80/10/10 partition with a fixed random seed, preventing overlap between the training and final evaluation problems.

We then generate 5 candidate solutions per problem with the base **Qwen2.5-Coder-1.5B-Instruct** model, execute them against the provided test suite, and benchmark passing solutions over 7 runs using the median wall-clock runtime. For **Control SFT**, we select the first passing candidate; for **Runtime-Aware SFT**, we keep the passing candidate with the lowest median runtime. The selected training targets are written as JSONL files for fine-tuning, with the current pipeline using `training/data/control.jsonl` and `training/data/runtime_aware.jsonl` as the default output paths.

C. Profiling Tools

We measure GPU utilization, SM efficiency, occupancy, kernel/CPU time split, and operator-level CUDA/CPU time using PyTorch Profiler with TensorBoard. Additionally, we use custom scripts to perform roofline analysis and benchmark serving using HuggingFace and vLLM. We use Weights & Biases to store summaries from each run by logging latency, throughput, and maximum CUDA memory.

D. Baseline Measurement

We establish a baseline using the original **Qwen2.5-Coder-1.5B-Instruct** model with no fine-tuning. The model generates 5 candidate solutions per problem across 6646 problems (33230 total candidates), producing a baseline Pass@1 of 36.9%. These metrics define the pre-optimization correctness baseline against which fine-tuned models are compared.

V. PERFORMANCE TUNING METHODOLOGY

A. Runtime-Aware Supervised Fine-Tuning

The core training-time optimization we use is runtime-aware supervised fine-tuning (RA-SFT). Rather than training on the first correct solution a model produces, RA-SFT selects the

fastest correct candidate as the training target. This introduces an explicit signal to train for runtime into the fine-tuning objective.

For each of the 6,646 problems, the base model generates 5 candidate solutions with a system prompt to focus on efficiency of the generated code. Each candidate is executed 7 times and the median runtime is recorded. Candidates that fail any test cases are discarded. Among the passing candidates for a given problem, the one with the lowest median runtime is selected as the training example. This produces a dataset of fastest and correct pairs that directly supervise the model toward efficient code.

A **Control SFT** variant is identically trained but targets the *first* passing candidate rather than the fastest, so we may later analyze the effects of emphasizing a runtime selection signal in the fine-tuning process between the Control SFT and RA-SFT.

Both variants are fine-tuned using QLoRA [6]. The base model is loaded at 4-bit NF4 quantization and has LoRA adapters with rank 16 and $\alpha = 32$ attached to all linear projection layers. Training uses TRL’s SFTTrainer for 3 epochs with a cosine learning rate schedule. QLoRA reduces peak GPU memory during training to under ~ 10 GB, making fine-tuning feasible on a single A100 without full-precision weight copies.

B. LoRA Adapter Merging for Deployment

After training, the LoRA adapters are merged into the base model weights using PEFT’s `merge_and_unload()`, producing a standard `AutoModelForCausalLM` checkpoint at BF16 precision. This step is required because vLLM does not accept `PEFT PeftModel` objects at serving time; merged weights load directly from disk like any HuggingFace checkpoint.

Merging introduces no additional parameters and no change to model behavior. The merged checkpoint occupies ~ 3 GB on disk, matching the footprint of the original base model and confirming that QLoRA training adds zero inference-time overhead.

C. Serving Benchmark Workflow

After merging the LoRA adapters into standalone checkpoints, we benchmark serving-time efficiency under four settings: Base SLM with HuggingFace, Base SLM with vLLM, Runtime-Aware SFT with HuggingFace, and Runtime-Aware SFT with vLLM. This setup lets us separate backend effects from fine-tuning effects.

We first run a small smoke test to verify checkpoint loading, logging, and output saving, then scale to the full benchmark. For each model/backend pair, we record average latency per prompt, throughput, peak CUDA memory, and GPU utilization. The aggregated comparison is saved to `outputs/serving_full_results.csv` for analysis and reporting.

D. vLLM Continuous Batching

The serving-time optimization replaces HuggingFace’s eager-mode `generate()` loop with vLLM [2]. At batch=8, HuggingFace operates at low arithmetic intensity (~ 8 FLOPs/byte), leaving the workload memory bound and limiting GPU utilization to approximately 37%.

On the contrary, vLLM improves decode efficiency through **PagedAttention**, which reduces KV-cache fragmentation, and **continuous batching**, which keeps more requests active during decoding. Together, these mechanisms increase the effective decode batch size and improve GPU utilization.

In our benchmark, GPU utilization increases from approximately 37% under HuggingFace to 97% under vLLM for the Base model and RA-SFT, while throughput rises from ~ 230 tokens/s to over $\sim 1,780$ tokens/s.

VI. EXPERIMENTAL RESULTS

A. Profiling Results

We profiled the baseline model and both the fine-tuned variants using the same settings. Operator-level profiling results show that linear layers and matrix multiplications are by far the majority of the operations found in the baseline model as well as in both of the fine-tuned variants of the model. This is also true for scaled dot product attention.

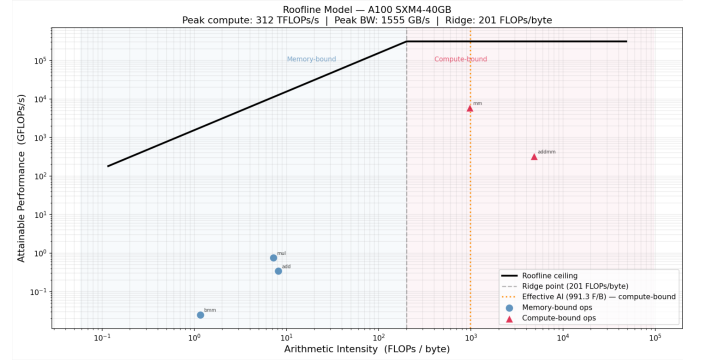


Fig. 1. Roofline Model for Base Model

TABLE I
TENSORBOARD PROFILING RESULTS ACROSS MODELS (BATCH_SIZE=64, A100)

Metric	Base SLM	Control SFT	Runtime-Aware SFT
GPU Utilization	73.51%	72.83%	68.31%
Est. SM Efficiency	67.7%	67.08%	62.93%
Est. Achieved Occupancy	49.76%	49.75%	49.68%
Kernel share of step time	74.93%	74.27%	69.59%
CPU Exec share of step time	17.98%	18.26%	22.04%
Tensor Core utilization	18.6%	18.6%	18.7%

	base ms	base %	control ms	control %	runtime_aware ms	runtime_aware %
Operator						
aten::linear	1203.62	3.24	1222.84	3.26	1242.36	3.28
cudaLaunchKernel	1166.64	3.14	1187.18	3.17	1207.50	3.19
aten::is_nonzero	724.94	1.95	726.43	1.94	719.45	1.90
aten::item	724.87	1.95	726.36	1.94	719.38	1.90
aten::local_scalar_dense	724.05	1.95	725.72	1.94	718.74	1.90
cudaStreamSynchronize	720.69	1.94	722.20	1.93	715.30	1.89
aten::scaled_dot_product_attention	580.42	1.56	580.46	1.55	588.01	1.55
aten::matmul	562.82	1.51	574.10	1.53	588.32	1.55
aten::mul	461.72	1.24	470.36	1.26	477.61	1.26
aten::mm	421.73	1.13	430.63	1.15	436.44	1.15
aten::copy_	388.56	1.04	393.35	1.05	399.16	1.05

Fig. 2. Operators Profiling Comparison

	base	control	runtime_aware
operator			
full_generate	2931.1248	2931.9705	2961.3712
cudaLaunchKernel	1578.3332	1577.1806	1288.1168
decode_remaining	1144.7180	1147.1743	1188.8811
aten::div	1097.8774	1097.5889	799.9699
aten::linear	365.2772	363.4080	656.4905
prefill	286.2018	280.3728	570.1023
aten::matmul	196.5367	194.5932	479.9639
aten::mm	154.1040	153.1217	436.8401
Command Buffer Full	268.3320	262.4251	203.2569
aten::scaled_dot_product_attention	202.5724	199.9547	207.0549

Fig. 3. Top 10 Operators By CPU Time (ms)

B. Performance Comparison

Tables II and III illustrate correctness and serving results across all model and backend configurations. Figure 4 breaks down observed GPU utilization into memory-bandwidth and compute activity. Figure 5 traces how output conciseness propagates into serving throughput.

TABLE II
CORRECTNESS AND HUGGINGFACE INFERENCE BASELINE (BATCH=8, A100)

Model	Pass@1	Latency/Prompt (s)	Throughput (tok/
Base SLM	34.5%	0.689	233.5
Control SFT	64.2%	0.696	221.9
Runtime-Aware SFT	63.9%	0.670	228.6

Both fine-tuned models achieve Pass@1 near 64% which is roughly $1.85\times$ the base model. RA-SFT matches Control SFT in correctness while producing slightly shorter outputs per request. All three models show HuggingFace GPU utilization near 37%, consistent with being memory-bound. The GPU is spending the majority of decode cycles stalled on HBM reads rather than executing Tensor Core instructions.

Switching to vLLM raises GPU utilization from $\sim 37\%$ to 97% for Base SLM and Runtime-Aware SFT, delivering $7.6\times$ and $8.3\times$ throughput gains respectively. Control SFT is the outlier with vLLM achieving only 78.0% GPU utilization

TABLE III
SERVING PERFORMANCE: HUGGINGFACE VS. vLLM (1000 PROMPTS, A100)

Model	Backend	Latency (ms)	Tok/s	GPU Util
Base SLM	HF	688.7	233.5	37.6%
	vLLM	31.8	1781.6	97.2%
Control SFT	HF	696.2	221.9	36.7%
	vLLM	39.4	1557.5	78.0%
Runtime-Aware SFT	HF	669.5	228.6	37.3%
	vLLM	31.7	1891.1	96.8%

and ~ 1557.5 tokens/s despite identical model weights and hardware. This is due to output length. Control SFT generates an average of 61.4 tokens per request versus 59.9 for Runtime-Aware SFT which occupies vLLM decode slots longer and reduces the number of concurrent sequences in flight. This is the efficiency cascade shown in Figure 5: the runtime-aware training objective produces more concise solutions, and that conciseness translates directly into a 21.4% throughput advantage (1891.1 vs. 1557.5 tok/s) over Control SFT at serving time.

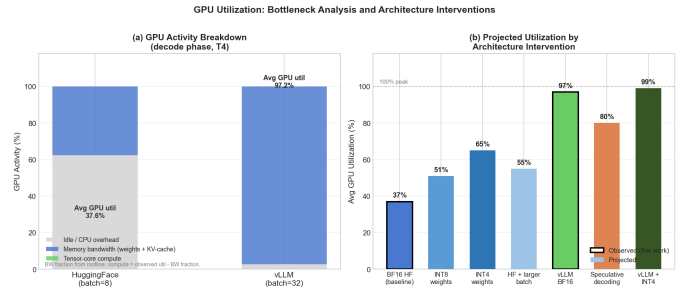


Fig. 4. (a) GPU activity breakdown for HuggingFace vs. vLLM decode. (b) Projected GPU utilization derived from roofline scaling.

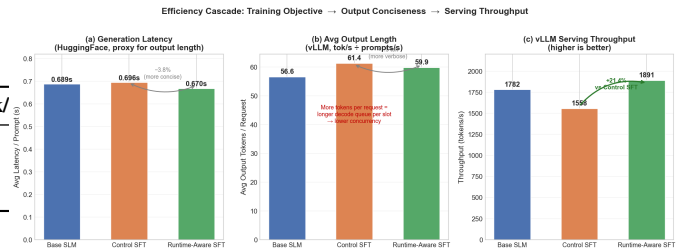


Fig. 5. (a) Generation Latency (Hugging Face). (b) Average output tokens per request (vLLM). (c) vLLM Throughput.

C. Profiling Analysis

a) Roofline Analysis:

The roofline model classifies each operator as either memory-bound or compute-bound relative to the A100 SXM4-40GB ridge point (i.e., ~ 201 FLOPs/byte), where peak computation is ~ 312 TFLOPS/s, and peak memory is 1555 GB/s, along

with the combined profiling of prefill and decode operations at the token level, with a total of 327 across 64 output tokens.

Fig 1 shows a bimodal structure. For reference, the two GEMM operators (`mm`, `addmm`) are located in compute-bound regions on the graph at AI $\sim 1,000$ – $4,000$ FLOPs/byte range. This is due to the prefill process of the prompt/sequence coming together at an astronomical arithmetic level and, therefore, the longer the sequence will be, the greater the AI, and thus AI settles above the ridge point. In contrast, the elementwise operators (`mul`, `add`) and smaller batch GEMM operators (`bmm`) are located in memory-bound regions on the graph with AI levels at ~ 1 – 10 FLOPs/byte. This is indicative of the decoding step for the autoregressive process of $M=1$ per token per pass; thus, no computation is required, but the weight matrix must be loaded from HBM each time a full batch is processed. Therefore AI levels for these operators are giving ~ 1 FLOPs/byte.

The aggregate effective AI is ~ 991 FLOPs/byte (dotted orange line), driven mainly by GEMM operators during the prefill phase, where prompt processing contributes most of the FLOPs in a single profiling pass. It should be noted that this metric is FLOPs-weighted rather than time-weighted. In production serving, end-to-end latency is dominated by the memory-bound decode phase of autoregressive generation.

b) *Hardware Efficiency:*

Table I shows that GPU utilization across the three models was relatively consistent, ranging from 71% to 73% (68% for the Runtime-Aware model). SM efficiency was also similar across models at approximately $\sim 68\%$. These values are moderate rather than high, which is expected for a memory-bound workload. A significant portion of runtime is spent waiting on HBM reads rather than executing instructions, leaving the GPU idle for much of the decode process.

The CPU/GPU Processor percent split shows the most accurate user perspective. Each of the three models was executed in the following manner: 74.9%, 74.3%, and 69.6% for GPU, and 18.0%, 18.3%, and 22.0% for CPU. The runtime-aware model incurred approximately an additional 4%-points of CPU-overhead than the other two models. The increase in this situation may have just as likely come from measurement variability rather than significantly altering the overall structure of the workload.

This level of CPU overhead is typical for HuggingFace eager-mode inference, where requests are processed largely independently. In contrast, vLLM reduces CPU overhead through continuous batching and PagedAttention, which improve GPU occupancy by serving multiple requests concurrently.

Across all three models, Tensor Cores utilization was approximately 18.6%–18.7% overall. Tensor Cores generally reaches maximum utilization when the GPU can process GEMM kernels continuously. Low levels of utilization here confirms that workloads are memory-bound. Therefore, at decode time ($M=1$), all GEMM kernels typically complete quickly, but each step is preceded by HBM weight loading, leaving Tensor Cores idle for much of the execution cycle.

CPU dispatch overhead further reduces the fraction of cycles in which Tensor Cores are active.

c) *Operator Breakdown:*

In Fig 2 and Fig 3, the operators show approximately similar percentage usages across all three models (within a margin of error of < 0.1 percentage point), supporting the conclusion that fine-tuning introduces no changes to the computational graph. `aten::linear` is the largest contributors to execution time, taking about 4.6% of the trace for each of the three models. However, there is no dominant operator for execution time; there is a load balancing of computing workload across hundreds of different lower-performing kernels, each contributing only a small fraction of total execution time.

About 3.7–3.8% of total runtime is accounted by the `cudaLaunchKernel` overhead. This was about equal to that of the largest compute operator. This indicates that there are a greater number of small kernels launched throughout the autoregressive decode process. By utilizing the combination of CUDA Graph capture and an improved serving framework such as vLLM, the dispatching overhead can be decreased.

The observations mentioned above lead to why vLLM has a higher throughput for serving purposes. Continuous batching provides an increase to the effective decode batch size, and PagedAttention allows more concurrent requests to reside in the GPU memory simultaneously. Together, these mechanisms increase throughput from 228.6 to 1891.1 tok/s in production serving workloads.

VII. LIMITATIONS

Marginal runtime improvement from training signal.

The distribution of the median execution times for all three models was virtually identical with Base SLM at 0.082 ms, Control SFT at 0.081 ms, and Runtime-Aware SFT at 0.081 ms. This means that the timing data for generating code has a minimal effect on the complexity of the algorithms. If the selection pool only contains five possible solutions for each problem, it is likely that none of the candidates will produce a statistically significant improvement in runtime, and the best candidate selected will usually be the same as the first correct candidate with only a small difference in runtime.

Approximate roofline analysis. Accurately calculating FLOPS and bytes transferred per operator on the hardware would require the use of NVIDIA Nsight Compute. Therefore, the operator level classification is intended to provide general guidelines about the operators’ expected classifications rather than provide exact classifications.

Single model family and scale. The experiments all use Qwen2.5-Coder-1.5B-Instruct. Thus, the results will vary with the size of the trained models and/or with different KV-cache configurations for different architectures.

Dataset domain. EffiCoder covers both algorithm and competitive-programming problems. Due to the fact that the experiment has not been generalized, the runtime selection criteria may be less informative for I/O-type or library-dependent code.

vLLM memory requirements. The total memory used by a vLLM on a GPU at high efficiency is $\sim 36\text{GB}$ vs. 4GB for HuggingFace. Therefore, the optimization of serving time cannot be achieved on GPUs having $< 40\text{GB}$ of VRAM without some form of quantization or model sharing.

Single hardware configuration. The profiling and serving benchmark tests were conducted on an A100-SXM4-40GB. Ridge points, utilization rates, and speedup ratios are variables that vary between hardware (e.g., T4, H100, VS cheap consumer with less memory bandwidth).

VIII. CONCLUSION

This project examined efficiency in training time and serving time for small code language models. The use of runtime-aware SFT improved the accuracy of the model to a degree; however, it did not significantly decrease the generated code's execution time. This is likely due to utilizing 5 candidates for each of the problems, where there is not enough variation among the choices that would provide a strong computed signal for these problems.

The serving optimization has a much greater effect on these smaller models. HuggingFace inference at batch size 8 is memory-bound, while vLLM increases GPU utilization to 97%. The throughput also improves by $7.6\times$ to $8.3\times$. Runtime-Aware SFT also produces slightly shorter outputs, giving a 21.4% vLLM throughput advantage over Control SFT.

Future work might examine broader potential candidate pools as well as iterative self-improvement methods to improve the training signal of runtime aware learning. Other areas of interest would include accurate profiling of hardware with NVIDIA Nsight Compute and inference time quantization (INT4/INT8). These areas of improvement could move the roofline ridge point closer to a trimetric view of the roofline at time of service when serving a large number of records or batch size.

REFERENCES

- [1] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic ai," 2025.
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
- [4] D. Huang, G. Zeng, J. Dai, M. Luo, H. Weng, Y. Qing, H. Cui, Z. Guo, and J. M. Zhang, "EffiCoder: Enhancing code generation in large language models through efficiency-aware fine-tuning," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. PMLR, vol. 267, 2025, pp. 26 058–26 076.
- [5] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.
- [6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [8] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [9] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Guo, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.
- [10] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [11] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.
- [13] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [14] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

APPENDIX A PROFILING METHODOLOGY NOTES

Note: Arithmetic intensity for GEMM operators uses an analytical read/write model; non-GEMM operators fall back to CUDA allocation as a proxy.

APPENDIX B TENSORBOARD PROFILING RESULTS

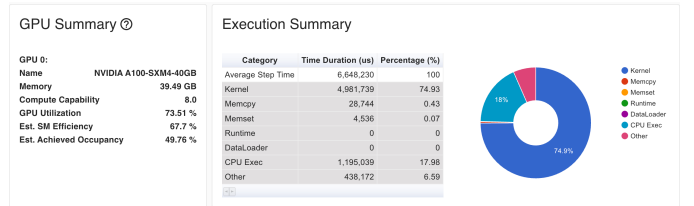


Fig. 6. Base Model TensorBoard Overview

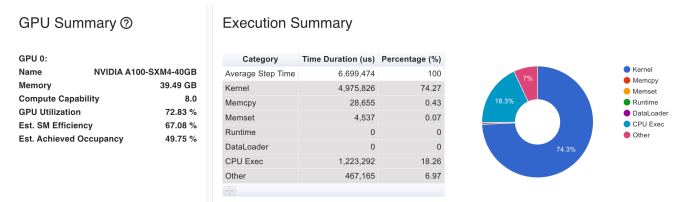


Fig. 7. Control Model TensorBoard Overview

